

Documento editable - Método de Bisección

Estudiante: Andrey Felipe Pinto Uribe

Código: 0192177

Archivo fuente: Metodo_Biseccion_0192177_Andrey_Felipe_Pinto_Urbe.py

Código Fuente en GitHub: https://github.com/pintodeveloper/numerical-analysis/tree/main/entregas_moodle/biseccion

1. Objetivo

El programa implementa el método de bisección para aproximar una raíz de una función continua en un intervalo cerrado $[a, b]$.

2. Funcionamiento general del programa

- La tabla muestra los valores del intervalo a y b , el punto medio c , el valor $f(c)$ y el error estimado.
- Cuando $f(c)$ cambia de signo con uno de los extremos, el programa conserva el subintervalo donde se encuentra la raíz.
- La raíz aproximada corresponde al último punto medio calculado cuando se cumple la tolerancia.
- La verificación $f(\text{raiz})$ debe quedar cercana a cero, lo cual confirma que la aproximación es coherente.

3. Pruebas o validaciones realizadas

- Se prueba una función continua $f(x) = x^3 - x - 2$.
- Se usa el intervalo $[1, 2]$, donde la función cambia de signo.
- Se valida la convergencia, la cantidad de iteraciones, el error final y la evaluación de la función en la raíz aproximada.

4. Evidencia del código fuente

A continuación se incluye el código fuente del archivo entregado.

```
1  """
2  Método de Bisección
3  Estudiante: Andrey Felipe Pinto Uribe
4  Código: 0192177
5
6  Archivo preparado para entrega en Moodle. Contiene la implementación,
7  un ejemplo de ejecución y validaciones básicas de entrada.
8  """
9
10 from dataclasses import dataclass
```

```

11 from typing import Callable, List, Optional
12
13
14 @dataclass
15 class IteracionBiseccion:
16     numero: int
17     a: float
18     b: float
19     c: float
20     fa: float
21     fb: float
22     fc: float
23     error: float
24
25
26 @dataclass
27 class ResultadoBiseccion:
28     raiz: float
29     iteraciones: int
30     convergio: bool
31     historial: List[IteracionBiseccion]
32
33
34 def biseccion(
35     funcion: Callable[[float], float],
36     a: float,
37     b: float,
38     tolerancia: float = 1e-6,
39     max_iter: int = 100,
40 ) -> ResultadoBiseccion:
41     if a >= b:
42         raise ValueError("El extremo izquierdo del intervalo debe ser menor que el derecho.")
43     if tolerancia <= 0:
44         raise ValueError("La tolerancia debe ser un número positivo.")
45     if max_iter <= 0:
46         raise ValueError("El número máximo de iteraciones debe ser positivo.")

```

```
47
48     fa = funcion(a)
49     fb = funcion(b)
50
51     if fa == 0:
52         return ResultadoBiseccion(raiz=a, iteraciones=0, convergio=True, historial=[])
53     if fb == 0:
54         return ResultadoBiseccion(raiz=b, iteraciones=0, convergio=True, historial=[])
55     if fa * fb > 0:
56         raise ValueError("La función debe cambiar de signo en el intervalo [a, b].")
57
58     historial: List[IteracionBiseccion] = []
59
60     for numero in range(1, max_iter + 1):
61         c = (a + b) / 2
62         fc = funcion(c)
63         error = abs(b - a) / 2
64
65         historial.append(
66             IteracionBiseccion(
67                 numero=numero,
68                 a=a,
69                 b=b,
70                 c=c,
71                 fa=fa,
72                 fb=fb,
73                 fc=fc,
74                 error=error,
75             )
76         )
77
78     if fc == 0 or error <= tolerancia:
79         return ResultadoBiseccion(
80             raiz=c,
81             iteraciones=numero,
82             convergio=True,
```

```

83             historial=historial,
84         )
85
86         if fa * fc < 0:
87             b = c
88             fb = fc
89         else:
90             a = c
91             fa = fc
92
93     return ResultadoBiseccion(
94         raiz=historial[-1].c,
95         iteraciones=max_iter,
96         convergio=False,
97         historial=historial,
98     )
99
100
101 def imprimir_tabla(historial: List[IteracionBiseccion]) -> None:
102     encabezado = f"{'i':>3} {'a':>12} {'b':>12} {'c':>12} {'f(c)':>14} {'error':>12}"
103     print(encabezado)
104     print("-" * len(encabezado))
105
106     for paso in historial:
107         print(
108             f"{paso.numero:>3} "
109             f"{paso.a:>12.6f} "
110             f"{paso.b:>12.6f} "
111             f"{paso.c:>12.6f} "
112             f"{paso.fc:>14.6e} "
113             f"{paso.error:>12.6e}"
114         )
115
116
117 def funcion_ejemplo(x: float) -> float:
118     return x**3 - x - 2

```

```

119
120
121 def main() -> None:
122     a = 1.0
123     b = 2.0
124     tolerancia = 1e-6
125     max_iter = 100
126
127     resultado = biseccion(funcion_ejemplo, a, b, tolerancia, max_iter)
128     error_final = resultado.historial[-1].error if resultado.historial else 0.0
129
130     print("Método de Bisección")
131     print("Función: f(x) = x^3 - x - 2")
132     print(f"Intervalo inicial: [{a}, {b}]")
133     print(f"Tolerancia: {tolerancia}")
134     print(f"Maximo de iteraciones: {max_iter}\n")
135
136     imprimir_tabla(resultado.historial)
137
138     print("\nResultado final")
139     print(f"Convergio: {resultado.convergio}")
140     print(f"Raiz aproximada: {resultado.raiz:.10f}")
141     print(f"Iteraciones realizadas: {resultado.iteraciones}")
142     print(f"Error estimado final: {error_final:.10e}")
143     print(f"Verificacion f(raiz): {funcion_ejemplo(resultado.raiz):.10e}")
144
145
146 if __name__ == "__main__":
147     main()

```

5. Evidencia de ejecución

La siguiente salida fue obtenida ejecutando el archivo Python de esta carpeta.

python Metodo_Biseccion_0192177_Andrey_Felipe_Pinto_Uribe.py

Método de Bisección

Función: f(x) = x³ - x - 2

Intervalo inicial: [1.0, 2.0]

Tolerancia: 1e-06

Maximo de iteraciones: 100

i	a	b	c	f(c)	error

1	1.000000	2.000000	1.500000	-1.250000e-01	5.000000e-01
2	1.500000	2.000000	1.750000	1.609375e+00	2.500000e-01
3	1.500000	1.750000	1.625000	6.660156e-01	1.250000e-01
4	1.500000	1.625000	1.562500	2.521973e-01	6.250000e-02
5	1.500000	1.562500	1.531250	5.911255e-02	3.125000e-02
6	1.500000	1.531250	1.515625	-3.405380e-02	1.562500e-02
7	1.515625	1.531250	1.523438	1.225042e-02	7.812500e-03
8	1.515625	1.523438	1.519531	-1.097125e-02	3.906250e-03
9	1.519531	1.523438	1.521484	6.221756e-04	1.953125e-03
10	1.519531	1.521484	1.520508	-5.178886e-03	9.765625e-04
11	1.520508	1.521484	1.520996	-2.279443e-03	4.882812e-04
12	1.520996	1.521484	1.521240	-8.289059e-04	2.441406e-04
13	1.521240	1.521484	1.521362	-1.034331e-04	1.220703e-04
14	1.521362	1.521484	1.521423	2.593543e-04	6.103516e-05
15	1.521362	1.521423	1.521393	7.795631e-05	3.051758e-05
16	1.521362	1.521393	1.521378	-1.273947e-05	1.525879e-05
17	1.521378	1.521393	1.521385	3.260816e-05	7.629395e-06
18	1.521378	1.521385	1.521381	9.934278e-06	3.814697e-06
19	1.521378	1.521381	1.521379	-1.402611e-06	1.907349e-06
20	1.521379	1.521381	1.521380	4.265829e-06	9.536743e-07

Resultado final

Convergio: True

Raiz aproximada: 1.5213804245

Iteraciones realizadas: 20

Error estimado final: 9.5367431641e-07

Verificacion f(raiz): 4.2658294048e-06

```
numerical-analysis > main > Metodo_Biseccion_0192177_Andrey_Felipe_Pinto_Urbe.py
Metodo de Bisección
Función: f(x) = x^3 - x - 2
Intervalo inicial: [1.0, 2.0]
Tolerancia: 1e-06
Maximo de iteraciones: 100

1      a      b      c      f(c)      error
-----
1      1.000000  2.000000  1.500000 -1.250000e-01  5.000000e-01
2      1.500000  2.000000  1.750000  1.609375e+00  2.500000e-01
3      1.500000  1.750000  1.625000  6.660156e-01  1.250000e-01
4      1.500000  1.625000  1.562500  2.521973e-01  6.250000e-02
5      1.500000  1.562500  1.531250  5.912253e-02  3.125000e-02
6      1.500000  1.531250  1.515625 -3.405380e-02  1.562500e-02
7      1.515625  1.531250  1.523438  1.225042e-02  7.812500e-03
8      1.515625  1.523438  1.519531 -1.097125e-02  3.906250e-03
9      1.519531  1.523438  1.521484  6.221756e-04  1.953125e-03
10     1.519531  1.521484  1.520508 -5.178880e-03  9.765625e-04
11     1.520508  1.521484  1.520996 -2.279443e-03  4.882812e-04
12     1.520996  1.521484  1.521240 -8.289059e-04  2.441406e-04
13     1.521240  1.521484  1.521362 -1.034331e-04  1.220703e-04
14     1.521362  1.521484  1.521423  2.593543e-04  6.103516e-05
15     1.521362  1.521423  1.521393  7.795631e-05  3.051758e-05
16     1.521362  1.521393  1.521378 -1.273947e-05  1.525879e-05
17     1.521378  1.521393  1.521385  3.260816e-05  7.629395e-06
18     1.521378  1.521385  1.521381  9.934278e-06  3.814697e-06
19     1.521378  1.521381  1.521379 -1.402611e-06  1.907349e-06
20     1.521379  1.521381  1.521380  4.265829e-06  9.536743e-07

Resultado final
Convergio: True
Raiz aproximada: 1.5213804245
Iteraciones realizadas: 20
Error estimado final: 9.5367431041e-07
Verificacion f(raiz): 4.2658294048e-06

Process finished with exit code 0
```

6. Explicación de la salida

La salida confirma que el programa se ejecuta sin errores. Los valores impresos corresponden al ejemplo de prueba definido en el método `main` .

- La tabla muestra los valores del intervalo a y b, el punto medio c, el valor f(c) y el error estimado.
- Cuando f(c) cambia de signo con uno de los extremos, el programa conserva el subintervalo donde se encuentra la raíz.
- La raíz aproximada corresponde al último punto medio calculado cuando se cumple la tolerancia.
- La verificación f(raiz) debe quedar cercana a cero, lo cual confirma que la aproximación es coherente.

7. Conclusión

El programa cumple el objetivo del método numérico, presenta una estructura clara, valida condiciones importantes de entrada y muestra resultados coherentes para el caso de prueba utilizado.